

Real-Time Network Topology Monitoring and Single-Point-of-Failure Detection for GNS3 Using REST API Automation and Graph Theory

Short name: GNS3 Live Topology & Resilience Dashboard

Project type	Network automation and monitoring tool
Platform	GNS3 2.2.54, local server, REST API v2
Language and libraries	Python 3, Flask, requests, NetworkX, vis-network
Demonstration lab	CriticalLinkDemo: 2 VPCS (PC1, PC2) and 2 Ethernet switches (Switch1, Switch2)
Date	22 September 2026

Note on the title. The word *Resilience* is used instead of *Health* because the tool reports device status and single-point-of-failure risk. It does not measure performance metrics such as traffic, latency or CPU load.

Abstract

This project is a small web dashboard that connects to the REST API of a running GNS3 server, reads the devices and cables of the current project every three seconds, and draws the topology live in a browser. Devices are coloured by their reported state (started or stopped). Using graph theory, the tool also finds every cable whose loss would split the network into disconnected parts (a *bridge*, or single point of failure) and shows it as a red dashed line. Cables that have a parallel backup are correctly treated as safe. The dashboard needs no packet capture and no extra GNS3 nodes. In tests on a four-device lab, every change made in GNS3 was noticed by the dashboard server within 3.1 seconds and drawn in the browser within roughly six seconds.

1. Introduction and Problem Statement

A network whose devices are all powered on is not necessarily resilient. A single cable with no backup can split a network in two while every device still shows as running. Finding such weaknesses by hand in GNS3 means inspecting each device and each cable. This project automates that check: it reads the topology through the GNS3 REST API and shows device status and structural weaknesses together in one live view.

2. Objectives

- Read the devices and cables of a running GNS3 project through its authenticated REST API.
- Show the topology live in a browser, using the same layout as the GNS3 canvas.
- Show each device's state (started or stopped) by colour.
- Detect and highlight critical links automatically, treating parallel cables as backups of each other.
- Report login, connection and project errors with clear messages, and never show stale data as if it were live.

3. System Architecture

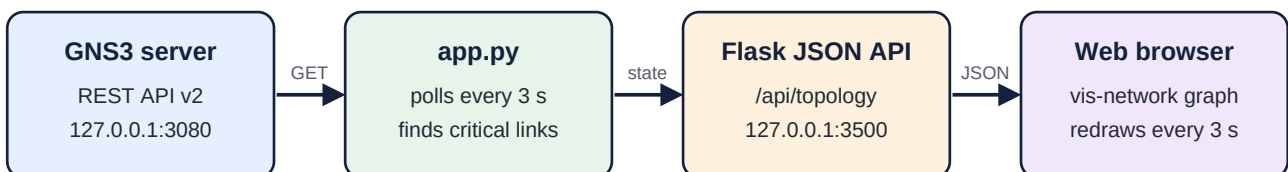


Figure 1. Data flow between GNS3, the Python application and the browser.

Data flow, step by step:

1. **Login.** `app.py` logs in to the GNS3 server at 127.0.0.1:3080 (HTTP Basic). The credentials come from the GNS3 GUI's own configuration on the same machine, so they are never typed, printed or stored.
2. **Read.** It finds the requested (open) project and requests its nodes and links every 3 seconds.
3. **Analyse.** It builds a graph with NetworkX and computes the set of critical links.
4. **Publish.** The result is served as JSON at `/api/topology` by Flask, on 127.0.0.1 only.
5. **Draw.** The browser requests the JSON every 3 seconds and redraws the graph with vis-network, using the device positions saved in GNS3.

4. Demonstration Lab

The lab is built only from GNS3's built-in device types, so no images need to be downloaded. The two parallel cables between the switches provide a backup path. Because the built-in switches do not run spanning tree, two parallel trunks form a layer-2 loop, so the demonstration never sends traffic between the PCs.

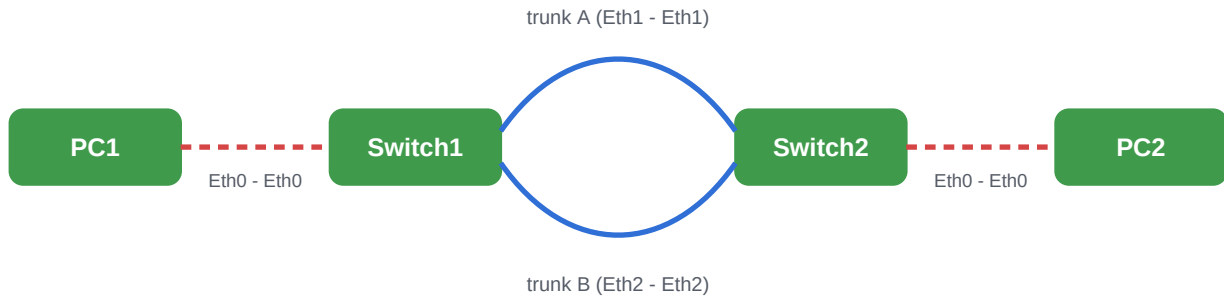


Figure 2. Lab topology. Red dashed = single cable with no backup; blue = cable with a backup.

Cable	From	To
1	PC1 Ethernet0	Switch1 Ethernet0
2	PC2 Ethernet0	Switch2 Ethernet0
3 (trunk A)	Switch1 Ethernet1	Switch2 Ethernet1
4 (trunk B)	Switch1 Ethernet2	Switch2 Ethernet2

5. Critical Link Detection

Devices are the vertices and cables are the edges of an undirected graph. A *bridge* is an edge whose removal increases the number of disconnected parts of the graph; NetworkX finds these with its `bridges()` function. NetworkX merges parallel edges into one, which would wrongly flag a cable that has a backup. The tool therefore counts how many physical cables join each pair of devices, and a bridge is reported as critical only if exactly one cable backs it.

Topology	Cables reported as critical
PC1, PC2 and two trunks (the lab)	The two PC cables only
The lab with one trunk removed	All three cables
Three switches in a triangle	None
Three switches in a line	Both links

Each PC has only one cable, so its cable is always critical. This is correct behaviour, not a fault: if that cable fails, the PC is cut off.

6. Implementation

Component	Technology
Network simulator	GNS3 2.2.54, REST API v2 (local server, port 3080)
Back end	Python 3, Flask 3.1, requests, NetworkX 3.6
Front end	HTML and JavaScript, vis-network 9.1.6 (loaded from a CDN)

File	Purpose
app.py	Polls the GNS3 API, computes critical links, serves the dashboard
gns3_client.py	GNS3 API client and credential helper shared by both scripts
setup_lab.py	Builds and starts the demonstration lab through the API
requirements.txt	Python dependencies

Design decisions

- **Polling every 3 seconds** keeps the code simple; no WebSocket connection is needed.
- **Dashboard port 3500.** GNS3 assigns console ports from TCP 5000 to 10000, so the dashboard stays outside that range to avoid taking a device's port.
- **Local only.** The dashboard binds to 127.0.0.1 because it has no login of its own.
- **Honest failure display.** On any error the graph is dimmed and labelled as the last known state.

7. How to Run

Run everything on the same Windows computer as GNS3, in PowerShell or Windows Terminal. Do not run it inside the GNS3 VM or in a device console.

One-time setup

```
pip install -r requirements.txt
python setup_lab.py --from-gns3-config
```

Each session

1. Open GNS3.
2. Open the project: File, Open project, CriticalLinkDemo.
3. Click the green Start all button in the toolbar.
4. Run the dashboard:

```
python app.py --project CriticalLinkDemo --from-gns3-config
```

5. Open `http://localhost:3500` in a browser. The page refreshes itself.

8. Results and Demonstration

The demonstration changes the lab in GNS3 and records what the dashboard shows. The screenshots below are real captures of the running dashboard (empty background trimmed).

Step	Action in GNS3	Dashboard result
1	None (baseline)	4/4 nodes started, 4 links, 2 critical
2	Stop PC2	PC2 turns red; 3/4 nodes started
3	Delete one Switch1-Switch2 cable	Every cable turns red dashed; 3 links, 3 critical
4	Add the cable back	Trunks blue again; 4 links, 2 critical

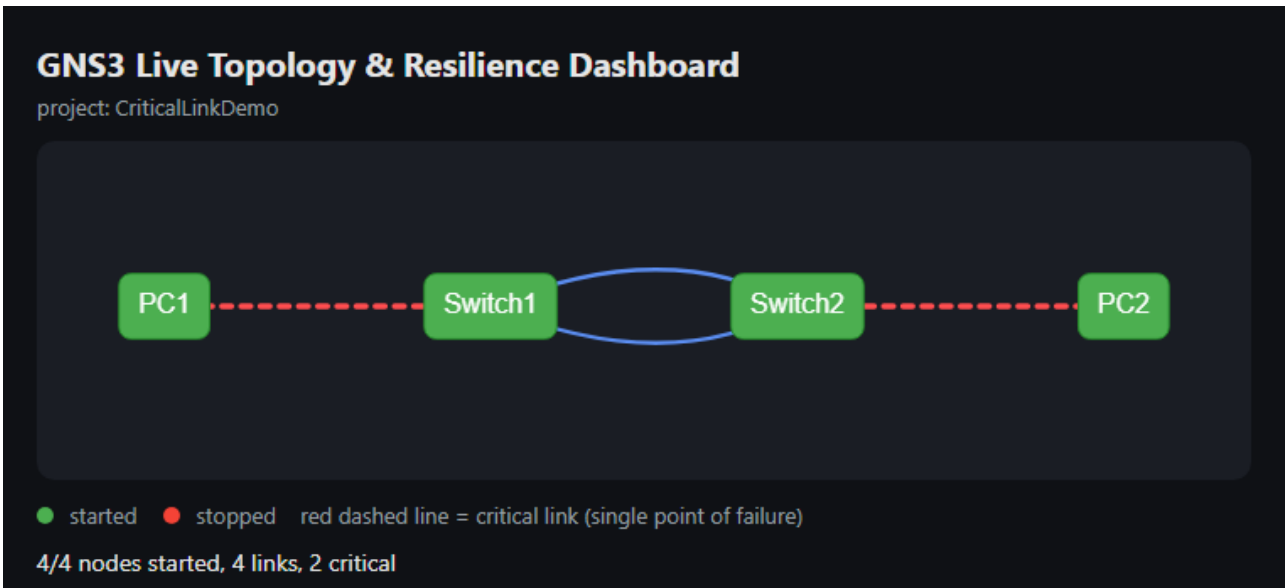


Figure 3. Baseline. All devices started; the two blue curves are the parallel trunks; the red dashed lines are the single PC cables.

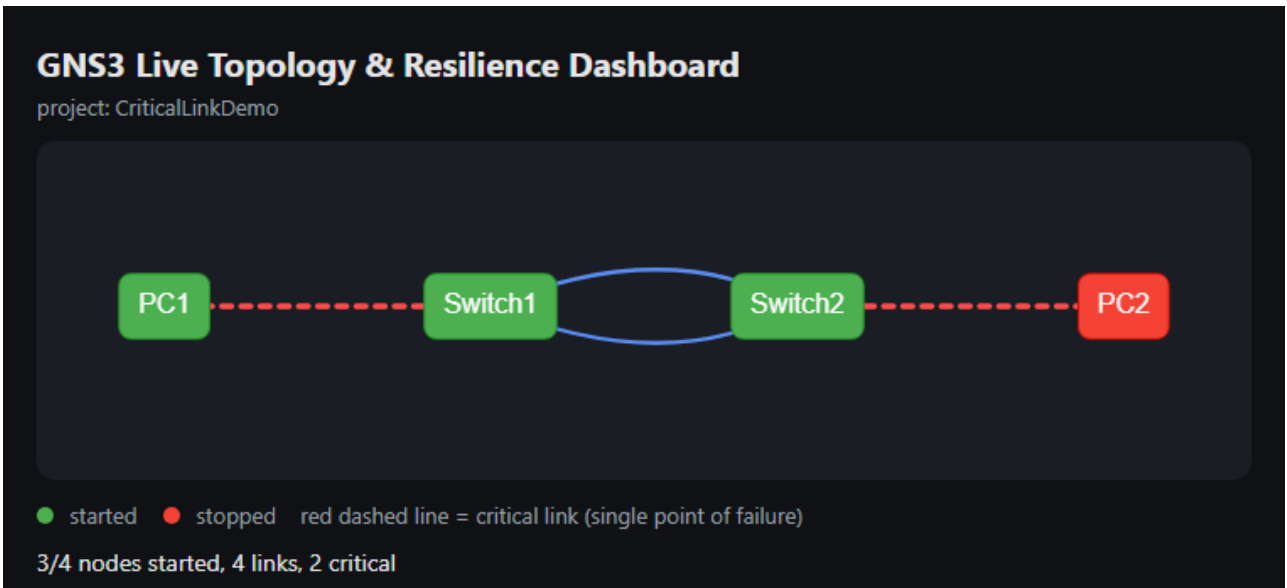


Figure 4. PC2 stopped. It turns red and the footer shows 3/4 nodes started.

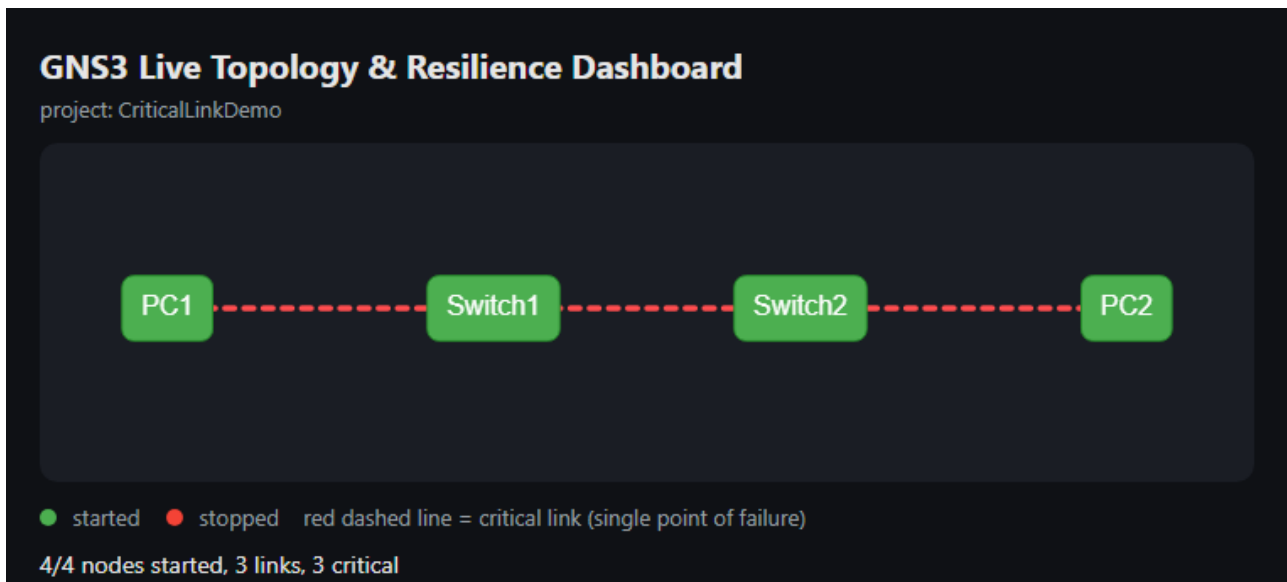


Figure 5. One trunk removed. Every cable is now critical, although all four devices are still green. A device-only status check would report this network as healthy.

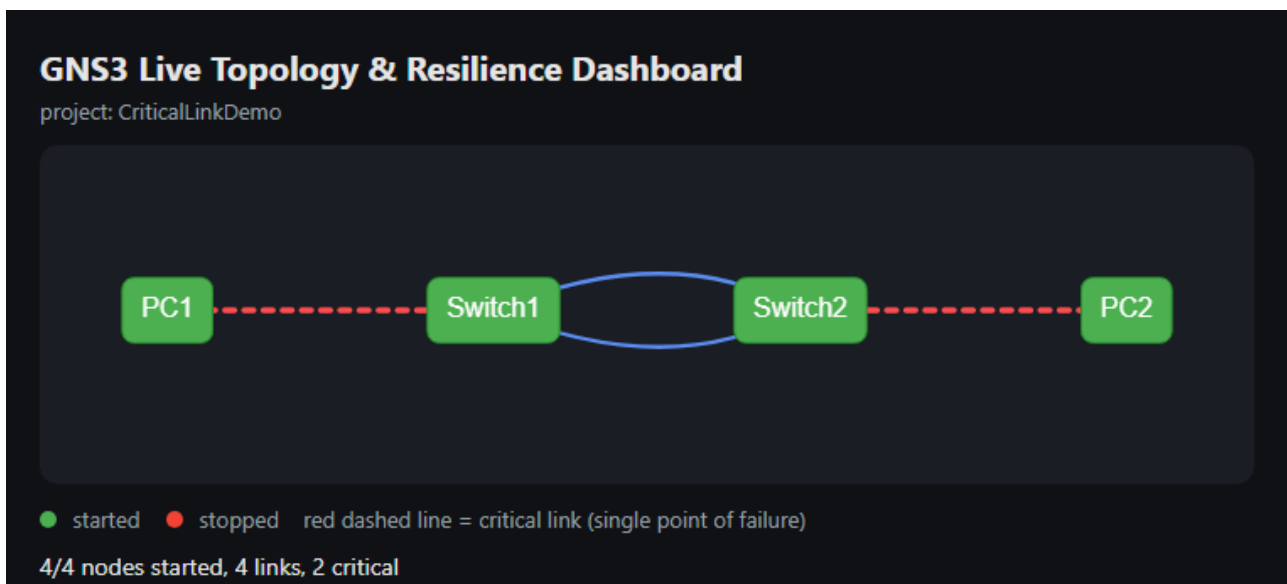


Figure 6. Trunk restored. The display returns to the baseline.

Timing. Across four runs (16 changes) the dashboard server noticed each change within 0.0 to 3.1 seconds. The browser adds up to 3 more seconds, so a change appears within about six seconds.

9. Validation and Error Checking

Check	Result
Syntax of all Python files	Passed
Dependency consistency (pip check)	No broken requirements
Static analysis (pyflakes)	No issues
Server log review (2,830 requests)	All successful; only two favicon 404 responses, since removed
Wrong password	"GNS3 rejected the credentials (HTTP 401)"
No credentials supplied	Same HTTP 401 message
Unknown project name	"No project named ..." with a hint
GNS3 not running or wrong port	"Cannot reach GNS3 at host:port -- is GNS3 running?"
Dashboard server stopped mid-session	Graph dims, labelled stale; recovers by itself

Issues found and fixed

- **Parallel cables merged.** The graph library collapsed the two trunks into one edge, wrongly flagging them. Cables are now counted by multiplicity.
- **Port collision.** The original dashboard port 5001 lies inside GNS3's console port range and collided with a device. The default is now 3500.
- **Stale data shown as live.** The graph is now dimmed and labelled when the connection is lost.
- **Inconsistent snapshots.** Server state was updated field by field; it is now published in one step.
- **Overlapping devices.** The automatic layout could hide a device; positions now come from GNS3.
- **Operational lesson.** Killing a device's `vpcs . exe` process by hand leaves GNS3 out of sync. Devices should be stopped and started from GNS3.

10. Limitations and Future Work

Limitations

- Only GNS3's reported status is read. Traffic, latency, CPU and interface errors are not measured.
- Critical links depend on the cabling only. A stopped device does not change which cables are critical.
- Suspended devices are shown in red, the same as stopped ones.
- GNS3's built-in Ethernet switches always report *started*, so only the PCs can turn red in this lab.
- One project is shown at a time, and updates arrive by polling rather than push.

Future work

- Remove stopped devices from the graph when computing critical links.
- Add ping-based reachability checks and an amber colour for suspended devices.
- Log status changes over time and chart uptime per device.
- Start and stop devices from the web page.

11. Conclusion

The project turns the GNS3 REST API into a live view that shows which devices are running and which cables are single points of failure. In the demonstration it reacted to every change within seconds and exposed a weakness that a device-only check cannot see.